Admin E2E 测试指南（PropOS）

> 适用版本：Vue 3 + Vite + Pinia + Element Plus（ `admin/` 目录）
>
> 文档日期：2026-04-28

一、测试分层架构

```
┌──────────────────────────────────────────────────────────────┐
│                    E2E 测试（Playwright）                     │
│  ┌──────────────────────────┐  ┌──────────────────────────┐  │
│  │ Mock 轨道                │  │ 真实后端轨道             │  │
│  │ page.route() 网络拦截    │  │ localhost:8080 透传      │  │
│  │ Chromium + Firefox       │  │ Chromium + Firefox       │  │
│  └──────────────────────────┘  └──────────────────────────┘  │
├──────────────────────────────────────────────────────────────┤
│            组件测试（@testing-library/vue + jsdom）           │
│         LoginView / store 联动 / form 渲染状态                │
├──────────────────────────────────────────────────────────────┤
│               Store 单元测试（@pinia/testing）                │
│     useAuthStore / useUsersStore / useAssetOverviewStore      │
├──────────────────────────────────────────────────────────────┤
│            API 层单元测试（axios-mock-adapter）               │
│              client.ts 拦截器 / Token 刷新链路                │
└──────────────────────────────────────────────────────────────┘
```

分层职责说明：

层级	职责	能力边界
E2E Mock	验证整体用户流程（真实浏览器 + 渲染 + 交互）	Element Plus 表单校验完整工作，弥补 jsdom 的 Transition stub 限制
E2E 真实后端	冒烟验证后端数据链路	需要本地后端服务运行
组件测试	验证组件渲染状态与 store 联动	仅能验证 `el-form-item--error` class，不能验证 `el-form-item__error` 文字
Store 单元	验证 action 逻辑与错误处理	快速、隔离、无渲染开销
API 单元	验证 axios 拦截器链路	最底层，与业务逻辑无关

二、E2E 技术栈

工具	版本	用途
`@playwright/test`	^1.50.0	测试运行器 + 浏览器控制
Chromium	内置	PC 后台主要浏览器（生产部署目标）
Firefox	内置	双浏览器兼容验证
`page.route()`	Playwright 内置	网络层 API 拦截（Mock 轨道）
`addInitScript`	Playwright 内置	在页面脚本执行前注入 token
`storageState`	Playwright 内置	登录态序列化与复用

三、目录结构

```
admin/
├── playwright.config.ts                    # Mock 轨道配置（5 个 project）
├── playwright.real.config.ts               # 真实后端轨道配置（3 个 project）
├── .env.e2e.example                        # 凭据配置模板（不提交 .env.e2e）
└── e2e/
    ├── global-setup.ts                     # 全局初始化：加载 .env.e2e 环境变量
    ├── auth.test.ts                        # 认证全链路（unauthenticated project 运行）
    ├── assets.test.ts                      # 资产管理（authenticated project 运行）
    ├── users.test.ts                       # 用户管理（authenticated project 运行）
    ├── setup/
    │   ├── auth.setup.ts                   # Mock 登录态初始化 → 写 .auth/user.json
    │   └── auth.real.setup.ts              # 真实登录初始化 → 写 .auth/real.user.json
    ├── real/
    │   └── smoke.test.ts                   # 真实后端冒烟测试
    ├── .auth/                              # 生成目录（gitignore）
    │   ├── user.json                       # Mock 登录态 storageState
    │   └── real.user.json                  # 真实登录态 storageState
    └── reports/                            # HTML 报告（gitignore）
```

四、Project 配置与运行流程

4.1 Mock 轨道（playwright.config.ts）

```
Phase 1  auth-setup (Chromium)
          ↓ 写 e2e/.auth/user.json
Phase 2  chromium-mock ──────────── assets.test.ts / users.test.ts
```

```
          (deps: auth-setup)
Phase 2b chromium-unauthenticated── auth.test.ts（无 storageState）
Phase 3  firefox-mock  ──────────── assets.test.ts / users.test.ts
          (deps: auth-setup)
Phase 3b firefox-unauthenticated ── auth.test.ts（无 storageState）
```

4.2 真实后端轨道（playwright.real.config.ts）

```
Phase 1  real-auth-setup (Chromium)
          ↓ 写 e2e/.auth/real.user.json
Phase 2  chromium-real ── real/smoke.test.ts
Phase 3  firefox-real  ── real/smoke.test.ts
```

五、Mock 策略

5.1 API 拦截（page.route）

```
// 在网络层拦截，CDP 协议捕获，绕过 Vite proxy 和真实后端
page.route('**/api/auth/login', route =>
  route.fulfill({
    status: 200,
    contentType: 'application/json',
    body: JSON.stringify({ data: { ...MOCK_TOKENS, user: MOCK_USER } }),
  })
)
```

响应格式严格遵循项目信封约定：

- 成功：`{ "data": <payload> }`
- 分页：`{ "data": [...], "meta": { "page": 1, "pageSize": 20, "total": N } }`
- 失败：`{ "error": { "code": "SCREAMING_SNAKE_CASE", "message": "中文描述" } }`

5.2 登录态注入（addInitScript）

```
// 在页面脚本执行前（DOMContentLoaded 之前）注入 token
// 确保 authGuard 在 beforeEach 时已能读到 token
await page.addInitScript(
  ([at, rt]) => {
    localStorage.setItem('access_token', at)
    localStorage.setItem('refresh_token', rt)
  },
  [MOCK_TOKENS.access_token, MOCK_TOKENS.refresh_token]
)
```

5.3 工厂函数模式

每个 API 场景封装为独立工厂函数，测试按需组合：

```
function mockLoginSuccess(page: Page) { ... }
function mockLoginFailure(page: Page) { ... }
function mockAssetsAll(page: Page) { ... }   // 组合多个路由 mock
```

六、测试场景清单

6.1 auth.test.ts（unauthenticated projects）

#	场景	关键断言	Mock
1	正确凭据登录	URL 跳转 `/dashboard`，`localStorage.access_token` 有值	login 200 + me 200
2	错误凭据登录	留在 `/login`，`.el-alert` 可见	login 401
3	空表单提交	`.el-form-item.el-form-item--error` 可见（真实浏览器校验）	无
4	格式错误邮箱 blur	邮箱 `el-form-item` 含 `el-form-item--error` class	无
5	未登录访问 `/assets`	URL 重定向 `/login`，含 `redirect=` query	无

> **Note**：场景 3、4 是 jsdom 单元测试无法覆盖的场景（Element Plus 的 Transition 在 jsdom 中成为 stub，校验文字不可见；real browser 中完整工作）。

6.2 assets.test.ts（authenticated projects）

#	场景	关键断言	Mock
1	三业态统计卡片渲染	"写字楼" 文字可见，出租率数字可见	overview + buildings + units
2	楼栋列表渲染	楼栋名称（"A栋写字楼"）在表格中可见	同上
3	点击楼栋 → 跳转详情	URL 变为 `/assets/buildings/bld-001`	overview + buildings + units + building detail + floors
4	楼栋详情页楼层列	"第1层" 在表格中可见	同上

6.3 users.test.ts（authenticated projects）

#	场景	关键断言	Mock
1	用户表格渲染	用户名（"张三"）在表格中可见	users 200 + departments 200
2	搜索参数传递	`waitForRequest` 捕获 URL 包含 `search=张三`	同上
3	角色过滤参数传递	`waitForRequest` 捕获 URL 包含 `role=leasing_specialist`	同上
4	分页器显示	`total=25` 时 `.el-pagination` 可见	users 200 (total: 25)

6.4 real/smoke.test.ts（real backend projects）

#	场景	关键断言	后端依赖
1	资产页真实数据加载	"资产管理" 标题可见，`.stat-card` 出现	GET /api/assets/overview 200
2	用户列表真实数据加载	"员工账号管理" 标题可见，`el-table` 出现	GET /api/users 200
3	网络响应状态验证	`/api/assets/overview` 响应状态为 200	同上

七、运行指南

7.1 首次安装

```
cd admin

# 安装 Playwright 依赖
pnpm add -D @playwright/test

# 安装浏览器（Chromium + Firefox）
pnpm exec playwright install chromium firefox

# 创建凭据文件（真实后端测试需要）
cp .env.e2e.example .env.e2e
# 编辑 .env.e2e，填写真实测试账号
```

7.2 日常运行

```
# Mock 轨道：全部场景（不需要启动后端）
pnpm test:e2e

# Mock 轨道：仅 Chromium（更快）
pnpm exec playwright test --project chromium-mock --project chromium-unauthenticated
```

```
# Mock 轨道：带 UI 调试界面
pnpm test:e2e:ui

# 查看 HTML 报告
pnpm test:e2e:report

# 真实后端冒烟（需先启动后端 localhost:8080）
pnpm test:e2e:real
```

7.3 调试技巧

```
# 暂停在指定断言（在测试中添加 await page.pause()）
# 然后用 headed 模式运行：
pnpm exec playwright test --headed --project chromium-unauthenticated

# 追踪失败录像（自动保存在 test-results/ 目录）
pnpm exec playwright show-trace test-results/*/trace.zip

# 单独运行某个测试文件
pnpm exec playwright test e2e/auth.test.ts

# 运行单个测试用例（按名称匹配）
pnpm exec playwright test -g "正确凭据"
```

八、配置说明

8.1 浏览器选择

Admin 是 PC 桌面端管理后台，使用 **Chromium + Firefox** 双浏览器覆盖：

- **Chromium**：Chrome/Edge 生产部署目标
- **Firefox**：跨浏览器兼容验证

视口统一为 `1440 × 900` （标准 PC 分辨率）。

8.2 webServer 配置

```
webServer: {
  command: 'pnpm dev',                // 启动 Vite 开发服务器
  url: 'http://localhost:5173',       // 等待此 URL 可用
  reuseExistingServer: !process.env.CI, // 本地开发复用已启动的服务器
  stdout: 'ignore',
  stderr: 'pipe',
}
```

> `reuseExistingServer: true` ：在本地已经 `pnpm dev` 的情况下，Playwright 不会重复启动服务器。CI 环境设为 `false` 确保每次都有干净的服务器状态。

8.3 环境变量

`.env.e2e`（真实后端测试使用，不提交到版本控制）：

```
E2E_USER_EMAIL=admin@propos.local
E2E_USER_PASSWORD=your_test_password
```

Mock 轨道不需要 `.env.e2e`，`global-setup.ts` 检测不到该文件时仅打印警告，不中断测试。

九、与单元测试的互补关系

测试维度	单元测试（jsdom）	E2E（Playwright）
Element Plus 表单 Error 文字	❌ Transition stub，不可见	✅ 真实渲染，完全可见
`el-form-item--error` class	⚠️ jsdom 中 validate() 链断裂	✅ 完整工作
HTTP 拦截	axios-mock-adapter（进程内）	page.route（网络层 CDP）
路由守卫（guard redirect）	✅ 可直接调用 authGuard 函数	✅ 真实浏览器导航触发
Store action 错误处理	✅ @pinia/testing 隔离	❌ 不适合（黑盒测试）
Token 刷新链路（并发）	✅ axios-mock-adapter 精确控制	❌ 难以可靠模拟
多浏览器兼容	❌ 仅 jsdom	✅ Chromium + Firefox
执行速度	极快（< 2s）	较慢（20-60s）

十、CI/CD 集成参考

```
# GitHub Actions 示例（.github/workflows/admin-e2e.yml）
- name: Install dependencies
  run: pnpm install
  working-directory: admin

- name: Install Playwright browsers
  run: pnpm exec playwright install --with-deps chromium firefox
  working-directory: admin

- name: Run E2E tests (mock)
  run: pnpm test:e2e
  working-directory: admin

- name: Upload E2E report
  uses: actions/upload-artifact@v4
  if: always()
```

```
  with:
    name: admin-e2e-report
    path: admin/e2e/reports/
```